

# Brief Technical Report – Property Data RAG System

Date: 2025-10-12

## 1) Goal and Scope

A Retrieval-Augmented Generation (RAG) system that answers natural-language questions about real estate properties using a large structured dataset (147k+ listings). The system supports both semantic retrieval and exact dataset-wide analytics for numeric questions, with a simple web UI and a REST API.

Core capabilities: - Document ingestion and vectorization of property records - Semantic retrieval with filters (price, beds, baths) - LLM-based answer generation with inline citations - Dataset-wide analytics for accurate aggregates (averages, comparisons, crime by area) - Conversational memory (multi-turn queries and refinements)

## 2) Architecture Overview

Frontend: Streamlit app (frontend/app.py) API backend: FastAPI (backend/main.py) Vector DB: ChromaDB persistent store (backend/vector\_store.py) Embeddings: Sentence-Transformers (all-MiniLM-L6-v2) LLM: Google Gemini (fallback to rule-based if no key) (backend/llm\_handler.py) RAG orchestration: (backend/rag\_pipeline.py) Global analytics: exact Pandas aggregations (backend/global\_analytics.py) Data ingestion & cleaning: (backend/data\_ingestion.py, scripts/load\_data.py) Conversation memory & analytics: (backend/conversation\_manager.py, backend/query\_analytics.py)

Data flow: CSV → Data cleaning → Documents(text+metadata) → Embeddings → ChromaDB ↑ ↓ Global analytics (Pandas) ← API ← RAG pipeline ← Frontend (Streamlit)

## 3) Data and Ingestion

Source: Property\_data.csv (147,668 rows) Required fields: address, price, bedrooms, bathrooms, type, listing date, description Optional fields: crime score, flood risk Ingestion: PropertyDataLoader loads/cleans data and creates a rich text representation + metadata per record; PropertyVectorStore batches documents into ChromaDB with Sentence-Transformers embeddings.

## 4) RAG Pipeline (Approach)

Query understanding: optional conversation context augments follow-ups Retrieval: semantic search in ChromaDB with optional numeric filters Exact analytics (when intent matches): average price for filtered cohorts top crime areas (by address or LAUA) with minimum listings type-to-type price comparisons Generation: LLM composes a concise, grounded answer; includes inline citation markers (e.g., "Guildford [1]") and a Sources block listing returned properties.

Contracts and success criteria: - Inputs: user query (+ optional filters, session id) - Outputs: answer text, top-K property metadata, filters applied, analytics\_used flag, citations, session id - Error modes: empty DB, invalid filters, LLM unavailable (fallback text) - Success: factual, grounded answers; filters respected; citations present; stable latency (<~3s typical with LLM)

## 5) Conversational Memory

ConversationManager stores sessions (disk persistence) and tracks filters, topics, previous queries. The pipeline enhances follow-up queries using prior context (types/locations/filters) and merges/retains filters when appropriate.

## 6) Key Components (Files)

backend/main.py: FastAPI app, startup wiring, endpoints (/query, /stats, /health, conversation routes)  
backend/vector\_store.py: ChromaDB persistent client, add/search with metadata filters  
backend/data\_ingestion.py: CSV load/clean; document text+metadata creation backend/rag\_pipeline.py: retrieval + analytics + LLM orchestration; inline citations backend/llm\_handler.py: Gemini integration; prompt building; fallback; citation appending backend/global\_analytics.py: Pandas-based exact aggregations and summaries frontend/app.py: Streamlit UI with filters, results, and conversation UX

## 7) Evaluation and Quality

Tests: basic import/init + conversation memory tests (tests/), all passing locally Analytics: /analytics endpoint exposes runtime stats; /health shows DB and analytics status Documentation: README.md, quick/complete setup guides; this brief report complements TECHNICAL\_REPORT.md

## 8) Deployment

Local: python scripts/load\_data.py → python backend/main.py → streamlit run frontend/app.py Docker: Compose file provided (see docs/DOCKER\_DEPLOYMENT.md)

## 9) Limitations and Next Steps

Intent detection is rule-based; can be improved with a lightweight classifier Type/location normalization can improve comparisons across synonyms (flat/apartment) Add authentication/rate-limiting for production APIs Optional: precompute analytics to JSON during data load to speed cold starts

## 10) Summary

The system cleanly separates ingestion, retrieval, analytics, and generation. Using ChromaDB and Sentence-Transformers keeps cost low and setup simple; Gemini provides high-quality responses with fallback resilience. The added global analytics ensures exact numeric answers, and inline citations increase transparency and trust.